Recommender Systems

Content-Based Recommendation

Video link: https://youtu.be/WsFnKXl2ixA

Thi Ngoc Trang Tran

Institute of Software Technology

Inffeldgasse 16b/II, 8010 Graz

trang.tran@tugraz.at

Example: Movie Recommendation

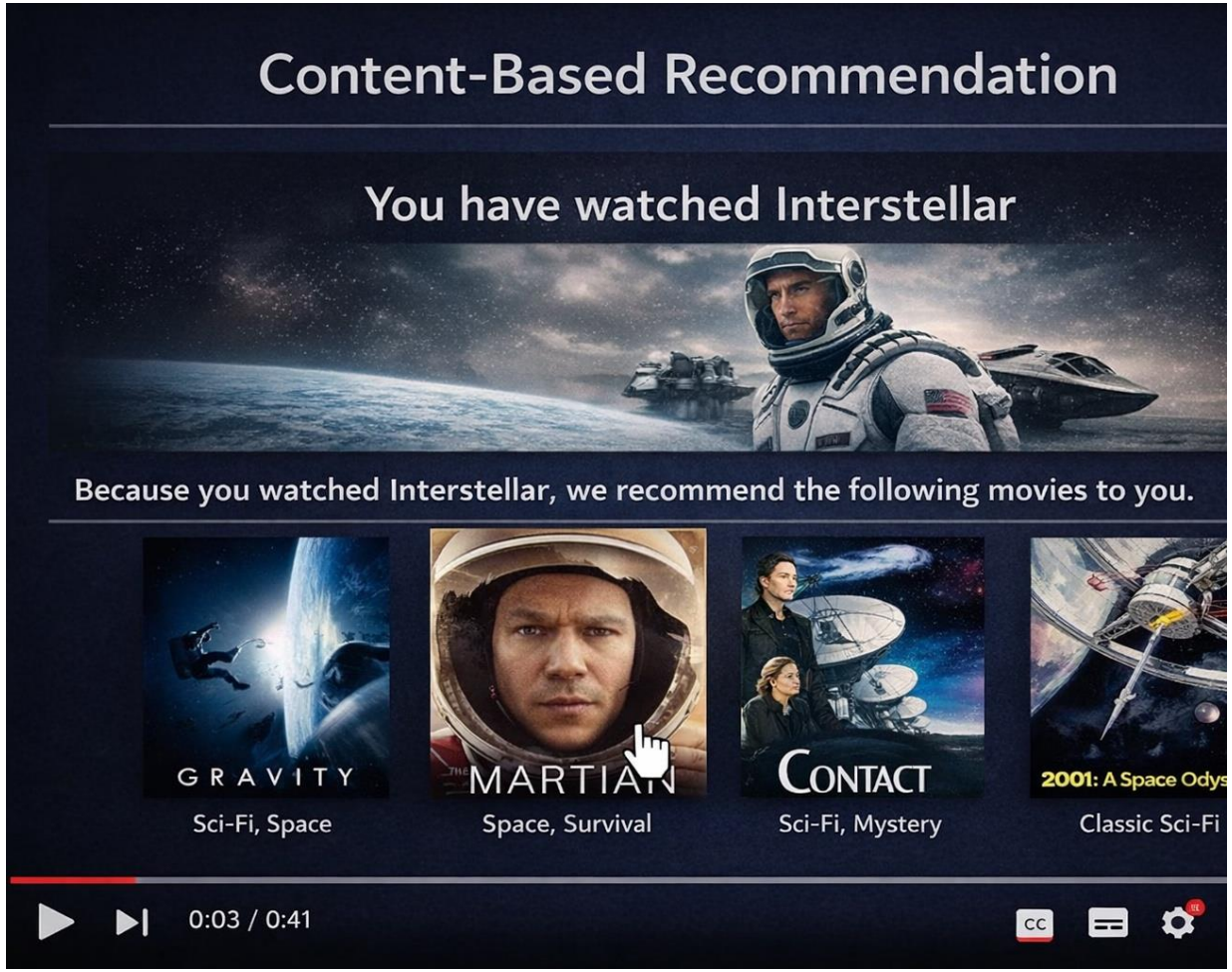
User watched and liked: **Interstellar**

- Extracted features: *Sci-Fi, Space, Survival, Future*

Candidate movies:

- Gravity
- The Martian
- Contact

Goal: Recommend movies with similar content

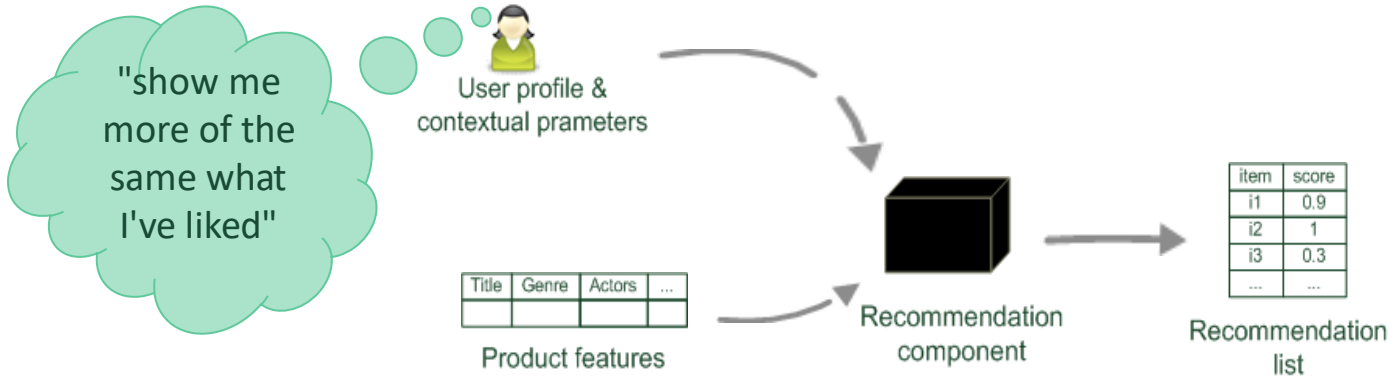


What we are going to discuss …

- Basic concepts
- The architecture
- Methods
  - Item representation:
    - Dice coefficient
    - Keyword-based vector space model
  - Learning user profiles:
    - Naïve Bayes classifiers
    - Relevance feedback method
    - Rocchio method
- Advantages and disadvantages

Content-based Recommender Systems

- **Content-based recommender systems** personalize suggestions using item descriptions and a user's past preferences.
- **Basic idea**: If a user has liked certain items, the system recommends similar items to him/her.
  - E.g., If a user has liked action movies, the system recommends other action movies to her.



What is the “Content”?

- Content: descriptive information describing items
- The content of items can be represented in two forms:
  - **Unstructured:** Basic characteristics of items are described by free-text descriptions
  - **Structured:** Each item is described by the same set of attributes



attributes

Title	Genre	Author	Type	Price	Keywords
The Night of the Gun	Memoir	David Carr	Paperback	29.90	Press and journalism, drug addiction, personal memoirs, New York
The Lace Reader	Fiction, Mystery	Brunonia Barry	Hardcover	49.90	American contemporary fiction, detective, historical
Into the Fire	Romance, Suspense	Suzanne Brockmann	Hardcover	45.90	American fiction, murder, neo-Nazism

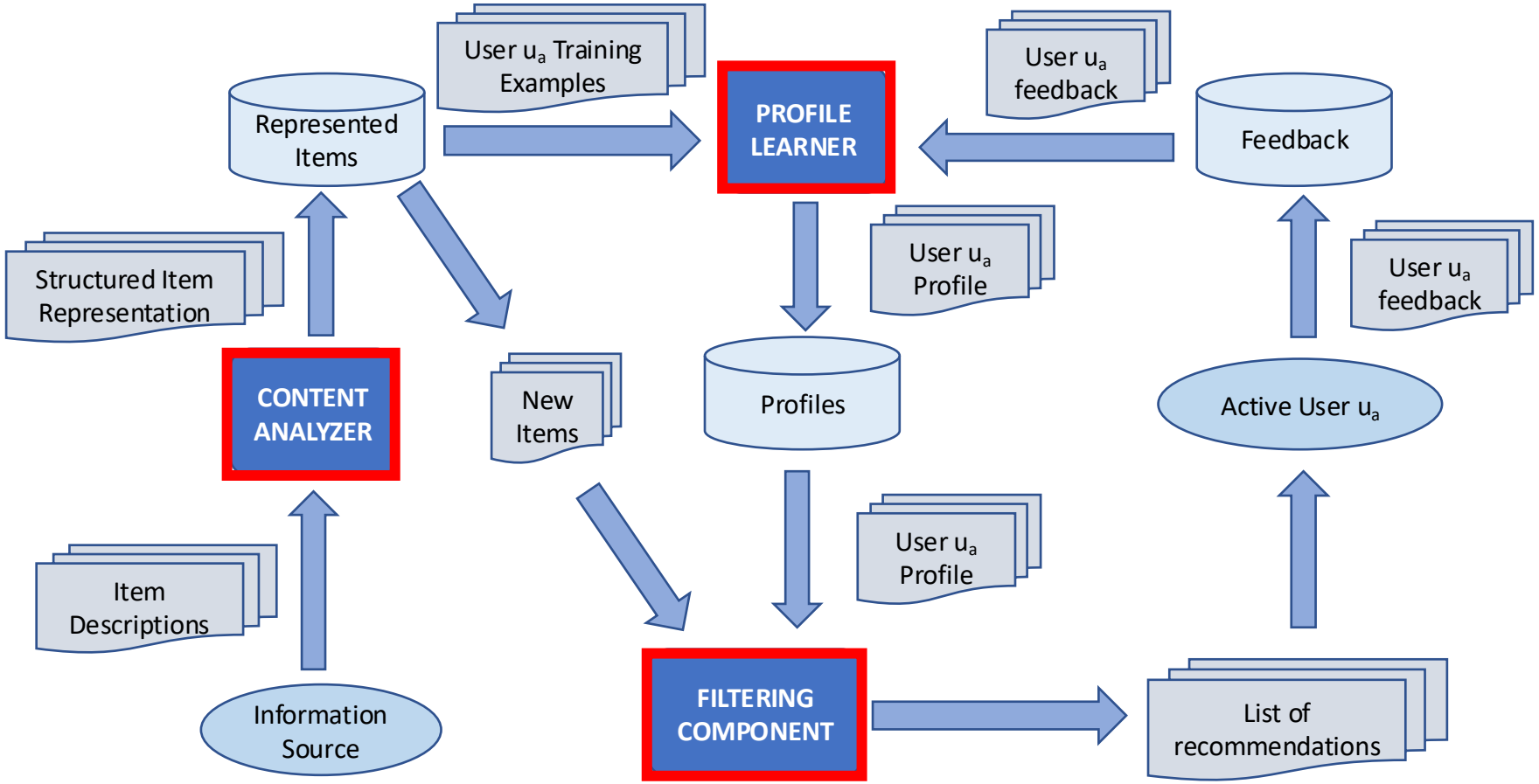
User Feedback (1)

- Two kinds of relevance feedback:
  - *Positive*: features liked by the user
  - *Negative*: features the user is not interested in
- Two techniques adopted for recording a user's feedback:
  - *Implicit*: derived from users' activities.
  - *Explicit:* a user evaluates items
    - Like/dislike
    - Ratings
    - *Text comments*

User Feedback (2)

- **Explicit feedback**:
  - Advantages: clear, simple, more user engagement
  - Disadvantages: cognitive burden, subjectivity and bias, lack of granularity (e.g., binary rating scales)
- **Implicit feedback**:
  - Advantages: reduced cognitive burden, abundance of data
  - Disadvantages:
    - noisy and biased data
    - user behavior may be influenced by other factors (e.g., interrupted by a phone call while reading) or situational factor (e.g., emotions)

A High-Level Architecture of A Content-based Recommender System



Source: **[LOP2011]**

Content Analyzer

- **Main tasks:** Extracts and represents item features
- **How?**
  - Transform raw data → structured feature vectors
  - Use feature extraction techniques
- **Types of raw data:**
  - **Structured data:** movie is described by predefined attributes
    - e.g., Intersteller → *Genre: sci-fi; Year: 2014. Director: Christopher Nolan, Duration: 169 mins*
  - **Unstructured data:** item description → keywords + weights (e.g., TF-IDF)
    - e.g., “space travel to save humanity” → *(space: 0.8, humanity: 0.6)*
- **Output:**
  - Feature vectors → used by Profile Learner and Filtering Component

Profile Learner

- **Main task**: build a model of the user's preferences / user profile
- **How?** Use machine learning techniques to learn a user interest model from liked/disliked items
- User profile is typically represented as a **feature vector**, **in the same space** as the item representations.
- **Example**: If a user likes *action and sci-fi* movies (e.g., *Inception* and the *Matrix*) and dislikes *romance* movies (e.g., Titanic)
  - Relevance feedback:
    - The user *prefers action and sci-fi, avoids romance*
    - User profile: action (+), sci-fi (+), romance (-)

Filtering Component

- **Main task**: Exploits the user profile to suggest relevant items to the user.
- **How?**
  - It computes a **similarity score** between the user profile and each item.
  - The higher the similarity, the more likely the item matches the user's preferences.
- **Output**:
  - Binary decision: relevant or not relevant
  - Ranked list: items ordered by predicted interest

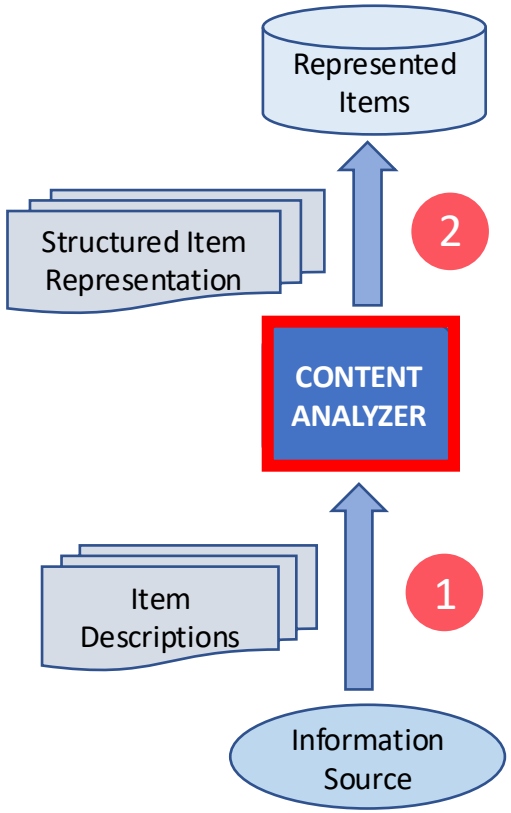
Recommendation Process

- **Step 1 –** Feature extraction
- **Step 2 –** User profile construction
- **Step 3 –** Recommendation generation

Recommendation Process

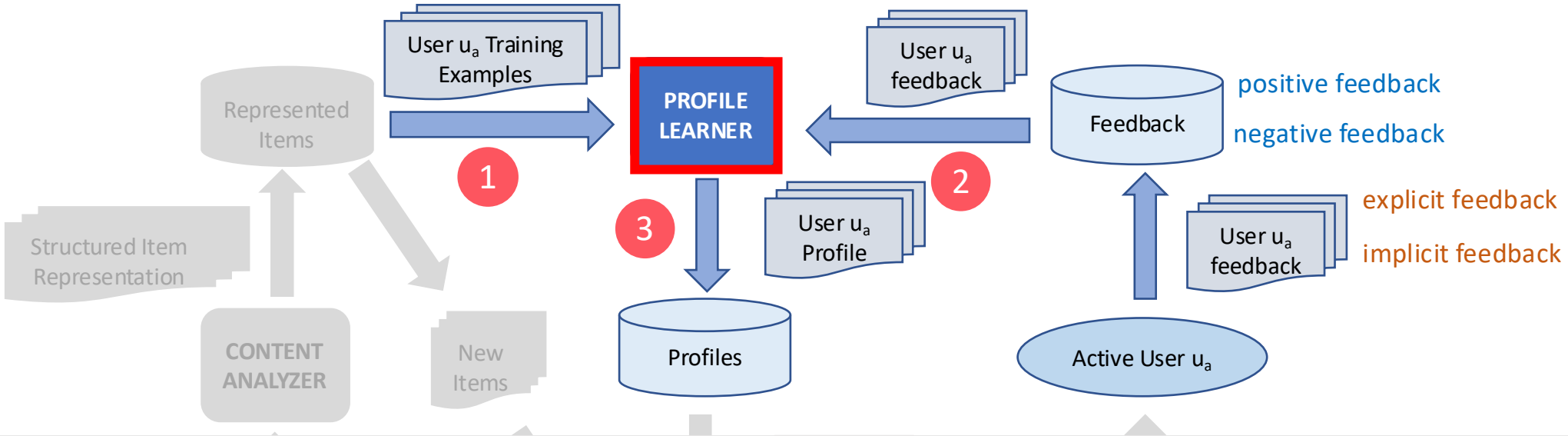
Step 1: Features Extraction

- Item descriptions are processed by *"Content Analyzer"*.
- Use Information Retrieval techniques to:
  - extract features from unstructured text to produce structured item representations
  - store item representations in the repository *"Represented Items"*



Recommendation Process

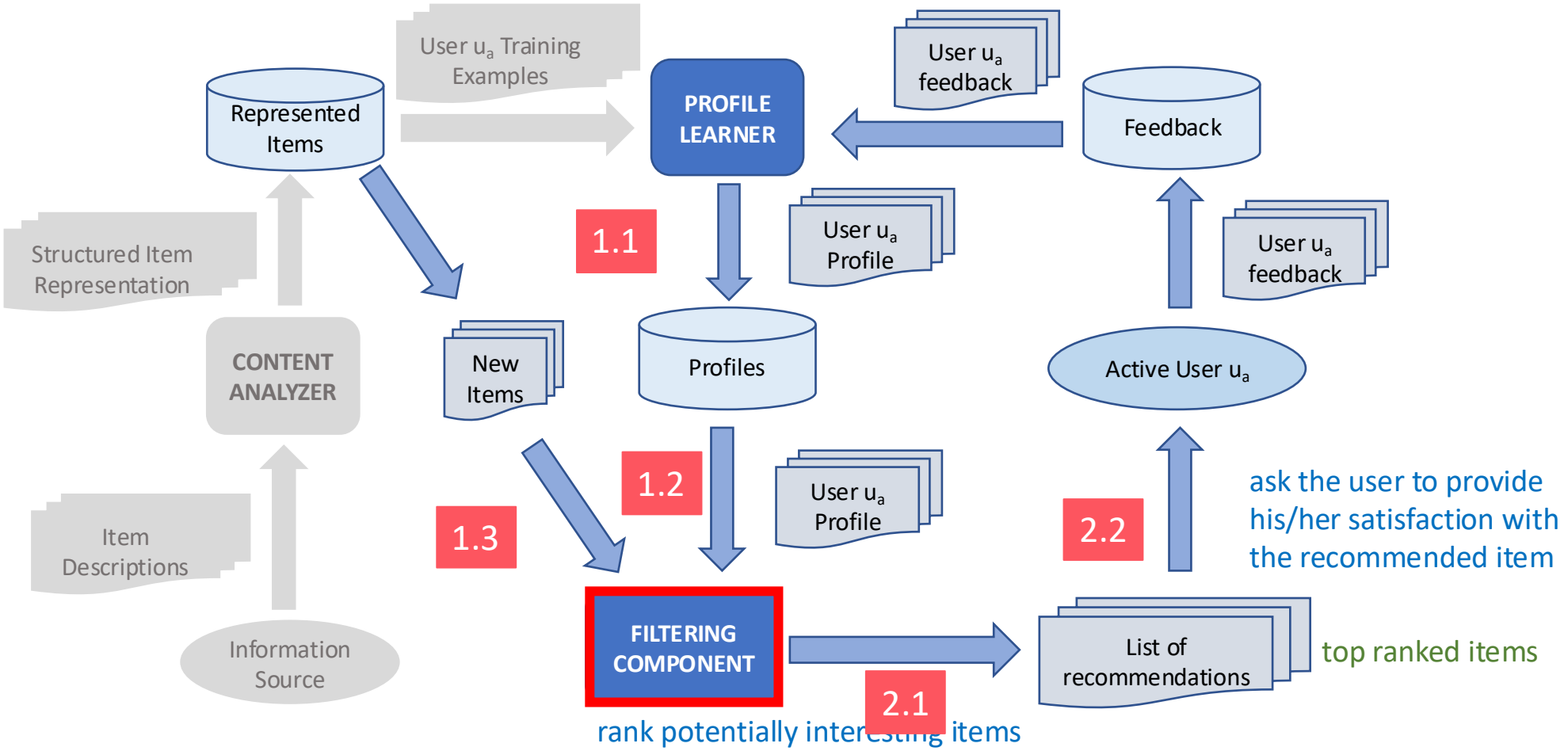
Step 2: User profile construction



- User profile is constructed in “Profile Learner”
- **How to build the profile of an active user u_a?**
  - Collect a training set of items and the user’s feedback on them.
  - A supervised learning algorithm (e.g., linear regression) uses this data to learn the user’s preferences and create a profile.

Recommendation Process

Step 3: Recommendation generation



Content-Based Recommendation Techniques

- **Item Representation approaches:**
  - Goal: *"What is the item?"*
  - represent items as feature vectors
- **Machine Learning approaches:**
  - Goal: *"Will the user like it?"*
  - learn user preferences from past interactions
  - predict if a new item is relevant

Content-Based Recommendation Techniques

- **Item Representation approaches:**
  - Goal: *"What is the item?"*
  - represent items as feature vectors
- **Machine Learning approaches:**
  - Goal: *"Will the user like it?"*
  - learn user preferences from past interactions
  - predict if a new item is relevant

Item Representation

- **Dice coefficient (*structured items*)**: Measures the similarity between the user profile and item description
- **Keyword-based Vector Space Model (*unstructured items*)**:
  - analyzes the textual content of items and user preferences
  - calculates the similarity between them
  - generates personalized recommendations

Dice Coefficient

- Item representation

Title	Genre	Author	Type	Price	Keywords
The Night of the Gun	Memoir	David Carr	Paperback	29.90	Press and journalism, drug addiction, personal memoirs, New York
The Lace Reader	Fiction, Mystery	Brunonia Barry	Hardcover	49.90	American contemporary fiction, detective, historical
Into the Fire	Romance, Suspense	Suzanne Brockmann	Hardcover	45.90	American fiction, murder, neo-Nazism

- User profile

Title	Genre	Author	Type	Price	Keywords
…	Fiction	Brunonia, Barry, Ken Follett	Paperback	25.65	Detective, murder, New York

- **Approach**: Compute the similarity between an unseen item and the user profile based on the category overlap (e.g. using the **Dice coefficient**)

$$similarity(U_a, item) = \frac{2 * |categories(U_a) \cap categories(item)|}{|categories(U_a)| + |categories(item)|}$$

Example: Learning Unit Recommendation (1)

learning unit (LU)	name	Java	UML	Management	Quality
l_1	Data Structures in Java	yes	yes		
l_2	Object Relational Mapping	yes	yes		
l_3	Software Architectures		yes		
l_4	Project Management		yes	yes	
l_5	Agile Processes			yes	
l_6	Object Oriented Analysis		yes	yes	
l_7	Object Oriented Design	yes	yes		
l_8	UML and the UP		yes	yes	
l_9	Class Diagrams		yes		
l_{10}	OO Complexity Metrics				yes

- Example set of software engineering related learning units (LU)
- Each of the learning units is additionally characterized by a set of categories

Example: Learning Unit Recommendation (2)

LU	rating (U_a)	name	Java	UML	Management	Quality	similarity(U_a, l_i)
l_1		Data Structures in Java	yes	yes			$\frac{2}{5}$ = 2*1 / 5 = 2 + 3
l_2		Object Relational Mapping	yes	yes			$\frac{2}{5}$
l_3		Software Architectures		yes			$\frac{2}{4}$
l_4	4.0	Project Management		yes	yes		–
l_5		Agile Processes			yes		$\frac{2}{4}$
l_6	4.0	Object Oriented Analysis		yes	yes		–
l_7		Object Oriented Design	yes	yes			$\frac{2}{5}$
l_8		UML and the UP		yes	yes		$\frac{4}{5}$
l_9		Class Diagrams		yes			$\frac{2}{4}$
l_{10}	3.0	OO Complexity Metrics				yes	–
user U_a				yes	yes	yes	

consumed by the user

- Example of content-based filtering: *user* U_a has already consumed the items l_4, l_6, and l_{10}.
- The item most similar to the preferences of U_a is l_8 and is now the best recommendation candidate for the current user.

$$similarity(U_a, item) = \frac{2 * |categories(U_a) \cap categories(item)|}{|categories(U_a)| + |categories(item)|}$$

Keyword-based Vector Space Model (VSM)

- VSM is a spatial representation of text documents.
- Every document is represented as a vector of term weights, where each weight indicates the degree of association between the document and the term.
  - $D = \{d_1, d_2, \dots, d_N\}$ is a set of documents
  - $T = \{t_1, t_2, \dots, t_n\}$ is the set of words in the corpus.
  - d_j is represented as a vector in a n-dimensional vector space, so $d_j = \{w_{1j}, w_{2j}, \dots, w_{nj}\}$, where w_{kj} is the weight for term t_k in document d_j.
- Document representation in the VSM raises two topics:
  - **weighting the terms**
  - **measuring the feature vector similarity**

Weighting the Terms Using TF-IDF (1)

- **TF-IDF (Term Frequency-Inverse Document Frequency)**:
  - specifies how important a word is for a document.
  - figures out which terms are the most relevant for a document.
- **TF (Term Frequency)** measures how often a term appears in a document.
- **IDF (Inverse Document Frequency)** reduces the weight of terms that appear in *ALL* documents.
- **TF-IDF** is based on an empirical observation: "*A term is likely to be relevant if it appears frequently in a document, but not frequently across all documents*".

Weighting the Terms Using TF-IDF (2)

- $TF(t_k, d_j)$:
  - the frequency of term t_k in document *dj.*
  - is *normalized* to prevent bias toward **long documents** (a term in a long document might have a higher count regardless of its importance).
  - $TF(t_k, d_j) = \frac{TF(t_k, d_j)}{TF_{max}}$, TF_{max} is the max frequency within the document.
- $IDF(t_k)$ is calculated as: $IDF(t_k) = log \frac{N}{DF_{t_k}}$
  - N is the number of all recommendable documents
  - DF_{t_k} is the number of documents from N, in which term t_k appears.
- $TF - IDF(t_k, d_j) = TF(t_k, d_j) * IDF(t_k)$

Weighting the Terms Using TF-IDF - Example

- Given a document d with terms *A*, *B*, and *C* with the following frequencies:
  - $TF(A,d) = 3;\ TF(B,d) = 2; TF(C,d) = 1$
- Assume document d belongs to a collection of 1000 documents. The document frequencies of the mentioned terms are:
  - $DF(A) = 50; DF(B) = 120; DF(C) = 98$
- $TF(t_k, d) = \frac{TF(t_k,d)}{TF_{max}}$; $TF_{max} = \max(TF(A,d), TF(B,d), TF(C,d)) = 3$
- $IDF(t_k) = log\ \frac{N}{DF_{t_k}}$
- $TF(A,d) = \frac{3}{3} = 1; \mathrm{IDF(A)} = \log\left(\frac{1000}{50}\right) = 1.3; \mathrm{TF-IDF(A}, d\ ) = 1.3$
- $TF(B,d) = \frac{2}{3} = 0.67; \mathrm{IDF(B)} = \log\left(\frac{1000}{120}\right) = 0.92; \mathrm{TF-IDF(B}, d) = 0.62$
- $TF(C,d) = \frac{1}{3} = 0.33; \mathrm{IDF(C)} = \log\left(\frac{1000}{98}\right) = 1.009; \mathrm{TF-IDF(C}, d\ ) = 0.33$
- Document $\boldsymbol{d} = (\mathbf{1.3}, \mathbf{0.62}, \mathbf{0.33})$, term A is the most relevant for document $\boldsymbol{d}$.

Measuring the Feature Vector Similarity

- The user profile and items are represented as weighted term vectors.
- The prediction of a user's interest in a particular document can be derived by computing the ***Cosine similarity***:

$$sim(d_j, P) = \frac{\sum_k w_{kj} P_k}{\sqrt{\sum_k {w_{kj}}^2} \cdot \sqrt{\sum_k {P_k}^2}}$$

the weight of term k in document dj

the weight of term k in the user profile P

- Items with **higher similarity scores** with the user profile are considered **more relevant and are ranked higher** in the recommendation list.

Content-Based Recommendation Techniques

- **Item Representation approaches:**
  - Goal: *"What is the item?"*
  - represent items as feature vectors
- **Machine Learning approaches:**
  - Goal: *"Will the user like it?"*
  - learn user preferences from past interactions
  - predict if a new item is relevant

Methods for Learning User Profiles

- A learning user profile problem:
  - refers to a binary text categorization task
  - classifies a document as "*relevant*" or "*not relevant*" to the user's preferences.
- Commonly used methods:
  - *Naïve Bayes and Probabilistic methods*
  - *Relevance feedback method*
  - *Rocchio method*

Naïve Bayes Classifier (1)

- generates a probabilistic model based on previously observed data.
- two classes: “relevant” and “not relevant”
- estimates a posteriori probability $P(c|d)$ of document d belonging to class c

$$P(c|d) = \frac{P(c)P(d|c)}{P(d)}$$

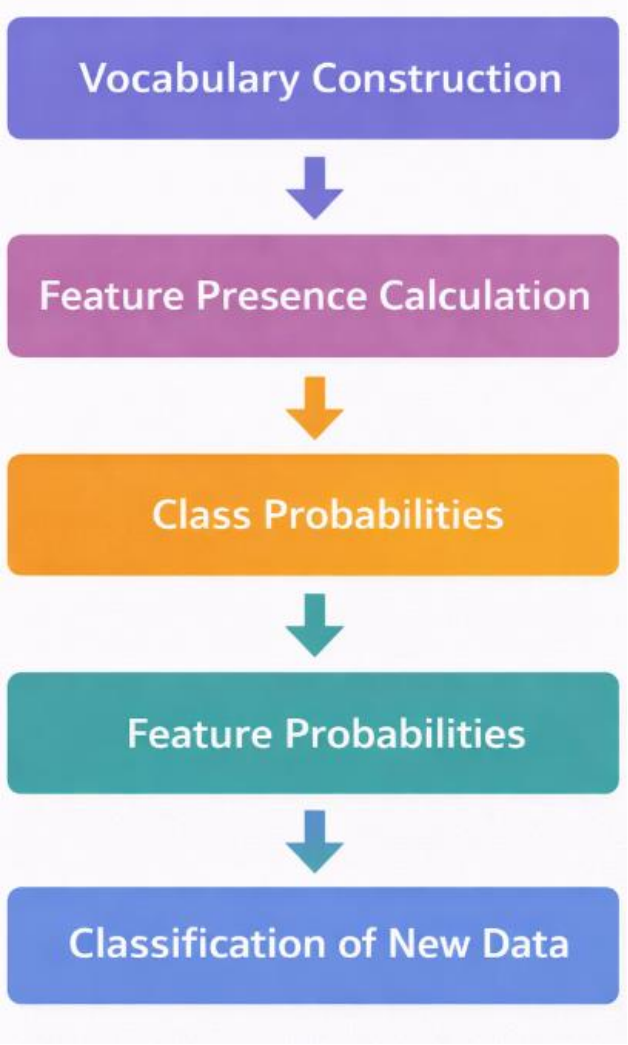
  - $P(c)$ is the probability of class c.
  - $P(d|c)$ is the probability of observing the document d given class c.
  - $P(d)$ is the probability of observing the document d.
- The document *d belongs to* class c with the highest probability:

$$c = argmax_{c_j} \frac{P(c_j)P(d|c_j)}{P(d)}$$

Naïve Bayes Classifier (2)

- There are models of Naïve Bayes Classifier:
  - **Multivariate Bernoulli event model** encodes each word as a binary attribute (i.e., 1: appeared/0: not appeared).
  - **Multinomial event model** counts how many times the word appeared in a document.

Multivariate Bernoulli Event Model – *Basic steps*



Step 1: Collect all unique terms from the training data.

Step 2: Represent each document as a binary vector (presence (1) and absence (0)).

Step 3: Calculate the prior probabilities of each class based on the training data.

Step 4: Calculate the probabilities of term t_k in each class. ***Laplace smoothing*** is applied to avoid zero probabilities ($\alpha = 1$).

$$P(t_k|c_j) = \frac{(\#occurrences\ of\ t_k\ in\ c_j + \alpha)}{(\#documents\ belonging\ to\ class\ c_j + \alpha * |vocabulary|)}$$

Step 5: Given a new document *D*, calculate its probabilities belonging to class c_j

$$P(c_j|D) = \frac{P(c_j) * P(D|c_j)}{P(D)} \qquad P(D|c_j) = \prod_{k=1}^{n} P(t_k|c_j) \qquad P(D) = \sum_{j=1}^{m} P(D|c_j) * P(c_j)$$

Multivariate Bernoulli Event Model - Example

- **Training data**:
  - Class = 1: relevant to the user
  - Class = 0: irrelevant to the user
  - Relevant documents: Doc_ids 1, 3, and 4
  - Irrelevant documents: Doc_ids 2 and 5
  - Number of relevant documents: 3
  - Number of irrelevant documents: 2
  - Total number of documents: 5
- **Our task**: *Is Document 6 relevant to the user?*

If a document is relevant to the user

Doc_id	recommender	intelligence	learning	school	class
1	1	1	1	0	1
2	0	0	1	1	0
3	1	1	0	0	1
4	1	0	1	1	1
5	0	0	0	1	0
6	1	1	0	0	?

new document

Multivariate Bernoulli Event Model – Steps 1, 2, & 3

- Step 1 – **Vocabulary construction:**
  - The vocabulary: “recommender”, “intelligence”, “learning”, and “school”.
- Step 2 – **Feature preference calculation**:
  - Documents belonging to the relevant class (+):
    - Document 1: [1, 1, 1, 0]
    - Document 3: [1, 1, 0, 0]
    - Document 4: [1, 0, 1, 1]
  - Documents belonging to the irrelevant class (-):
    - Document 2: [0, 0, 1, 1]
    - Document 5: [0, 0, 0, 1]
- Step 3 – **Class probabilities**:
  - P(relevant class) = P(+) = 3/5 = 0.6
  - P(irrelevant class) = P(-) = 1 - 0.6 = 0.4

Doc_id	recommender	intelligence	learning	school	**class**
1	1	1	1	0	1
2	0	0	1	1	0
3	1	1	0	0	1
4	1	0	1	1	1
5	0	0	0	1	0
6	1	1	0	0	?

Multivariate Bernoulli Event Model

Step 4 – Feature Probabilities

- **Relevant class (+)**: Use **Laplace Smoothing**, $\alpha = 1$
  - $P(\text{“}recommender\text{”}|+) = \frac{(3+1)}{(3+4)} = 0.571$
  - $P(\text{“}intelligence\text{”}|+) = \frac{(2+1)}{(3+4)} = 0.429$
  - $P(\text{“}learning\text{”}|+) = \frac{(2+1)}{(3+4)} = 0.429$
  - $P(\text{“}school\text{”}|+) = \frac{(1+1)}{(3+4)} = 0.286$
- **Irrelevant class (-):** Use **Laplace Smoothing**, $\alpha = 1$
  - $P(\text{“}recommender\text{”}|-) = \frac{(0+1)}{(2+4)} = 0.167$
  - $P(\text{“}intelligence\text{”}|-) = \frac{(0+1)}{(2+4)} = 0.167$
  - $P(\text{“}learning\text{”}|-) = \frac{(1+1)}{(2+4)} = 0.333$
  - $P(\text{“}school\text{”}|-) = \frac{(2+1)}{(2+4)} = 0.5$

Doc_id	recommender	intelligence	learning	school	class
1	1	1	1	0	1
2	0	0	1	1	0
3	1	1	0	0	1
4	1	0	1	1	1
5	0	0	0	1	0
6	1	1	0	0	?

$$P(t_k|c_j) = \frac{(\#occurrences\ of\ t_k\ in\ c_j + \alpha)}{(\#documents\ belonging\ to\ class\ c_j + \alpha * |vocabulary|)}$$

Multivariate Bernoulli Event Model

Step 5 – Classification new document

- **New document:** Document 6 [1, 1, 0, 0]

Doc_id	recommender	intelligence	learning	school	class
6	1	1	0	0	?

- $P(Document\ 6|+) = P(\text{“}recommender\text{”}|+) * P(\text{“}intelligence\text{”}|+) = 0.571 * 0.429 = 0.245$
- $P(+) = 0.6$
- $P(Document\ 6|-) = P(\text{“}recommender\text{”}|-) * P(\text{“}intelligence\text{”}|-) = 0.167 * 0.167 = 0.028$
- $P(-) = 0.4$
- $P(Document\ 6) = \big(P(Document\ 6|+) * P(+)\big) + \big(P(Document\ 6|-) * P(-)\big) = 0.158$
- The posteriori probability of Document 6 belonging to the relevant/irrelevant class:
  - $P(+|Document\ 6) = \frac{P(+)*P(Document\ 6|+)}{P(Document\ 6)} = \mathbf{0.930}$
  - $P(-|Document\ 6) = \frac{P(-)*P(Document\ 6|-)}{P(Document\ 6)} = 0.012$

$$P(c_j|D) = \frac{P(c_j) * P(D|c_j)}{P(D)}$$

Conclusion: *Document 6* is relevant to the user.

Multinomial Event Model
Basic steps

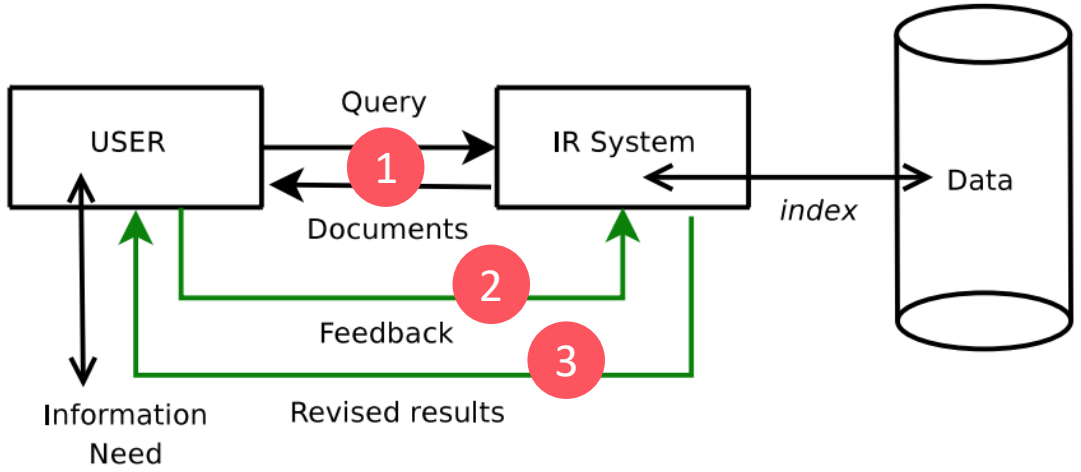
- **Steps 1, 2, 3, and 4: Similar to the corresponding steps in** *Multivariate Bernoulli Event Model*
- **Step 5 - Classification of New Data**: Given a new document D, calculate the probabilities of it belonging to a class c_j using the feature probabilities obtained in the previous step.

$$P(c_j|D) = \frac{P(c_j) * \prod_{k=1}^{n} P(t_k|c_j)^{N(D,t_k)}}{P(D)}$$

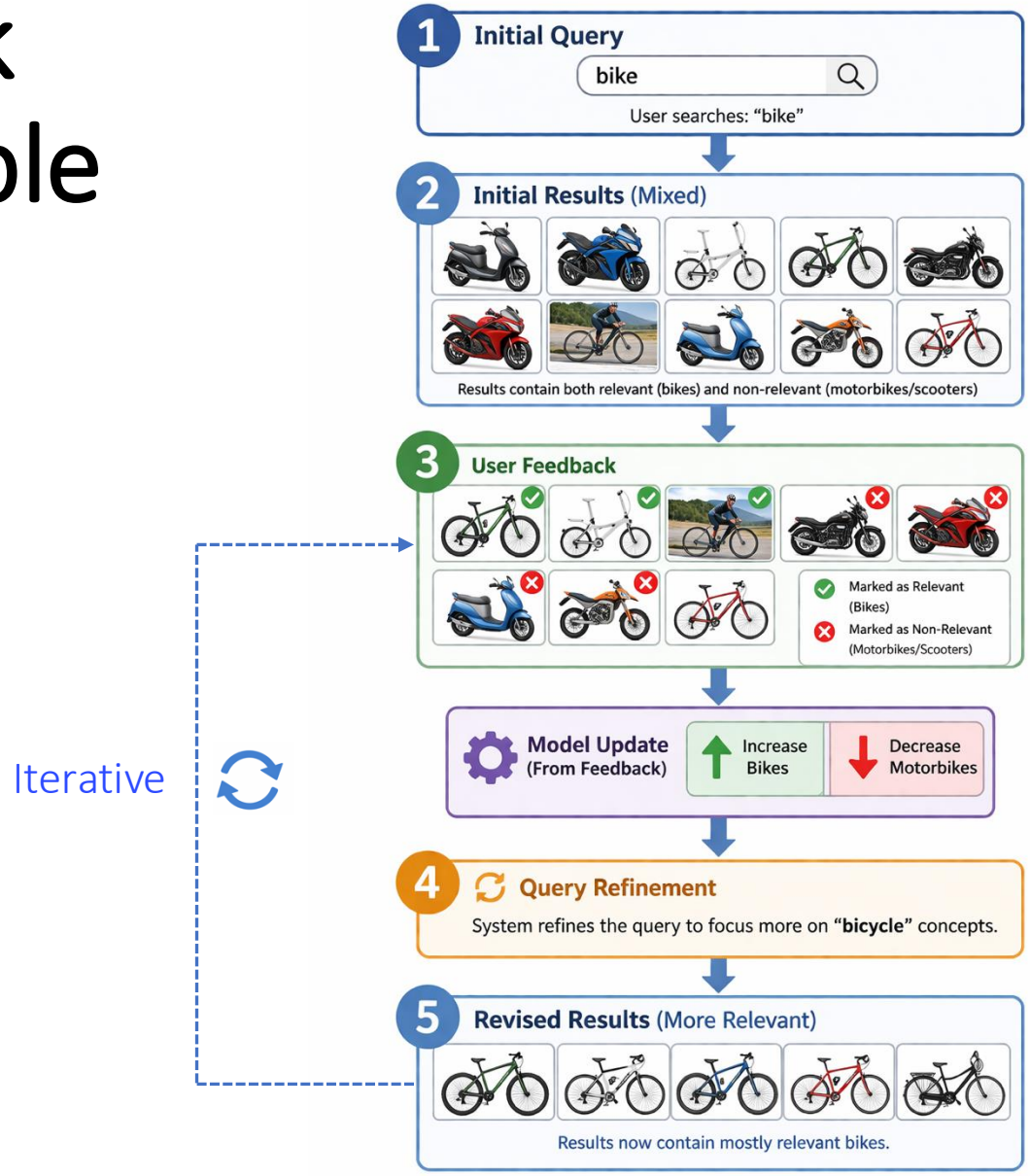
$\boldsymbol{N(D, t_k)}$ is the defined as the number of times term t_k appeared in document D.

Relevance Feedback Method

- Retrieval quality depends on the capability of users to formulate queries with right keywords.
- The query can result in *relevant* and *non-relevant* documents.
- **To increase the number of relevant document, users provide feedback on recommended items:**
  - Initial recommendations
  - User feedback
  - Revised results/recommendations
  - Iterative process



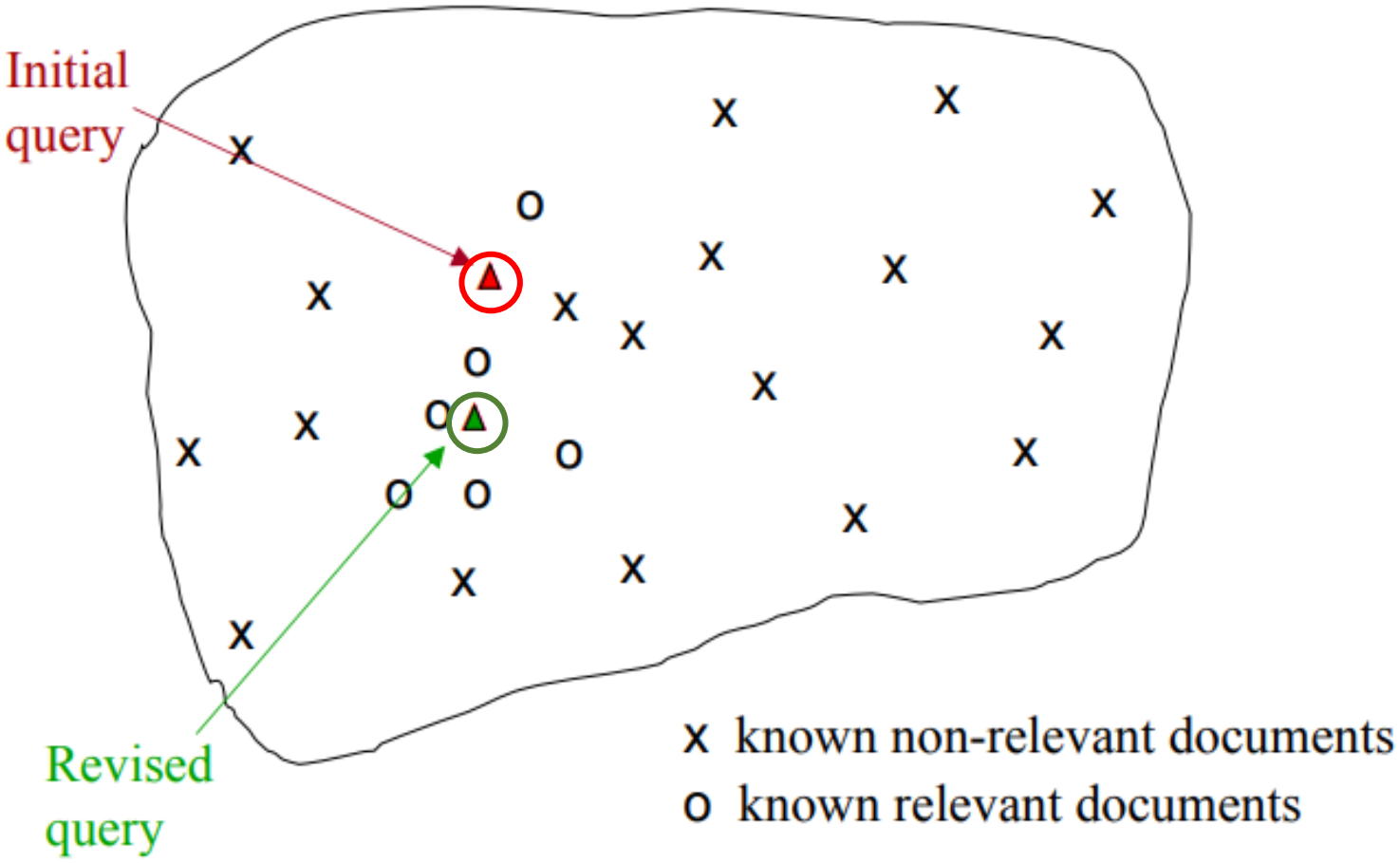
Relevance Feedback Method – An Example



Rocchio Method (1)

- Standard algorithm for the relevance feedback approach
- Integrates a measure of relevance feedback into the Vector Space Model
- Main idea: **Updates a query vector that**
  - *maximizes* the similarity with relevant documents.
  - *minimizes* the similarity with non-relevant documents.

Rocchio Method (2)



Rocchio Method (3)

- Computing modified query Q_{i+1} from the current query Q_i with:

$$Q_{i+1} = \alpha * Q_i + \beta \left(\frac{1}{|D^+|} \sum_{d^+ \in D^+} d^+ \right) - \gamma \left(\frac{1}{|D^-|} \sum_{d^- \in D^-} d^- \right)$$

- α, β, γ used to fine-tune the query:
  - α is the weight of original query Q_i
  - β is the weight of relevant documents
  - γ is the weight of irrelevant documents
- Reasonable values: $\alpha = 1.0$, $\beta = 0.75$ and $\gamma = 0.25$

- $|D^+|$ is the number of relevant documents
- $|\mathrm{D}^-|$ is the number of irrelevant documents
- **Notes:** If Q_{i+1} has negative term weights, then set them to zero.

Rocchio Method - Example

1) Initial Query

Query: "*recommender systems*"

Term	recommender	system	user	data
q_0	1.0	0.8	0.0	0.0

2) User Feedback

Relevant documents (liked)

Doc	recommender	system	user	data
d_1	0.9	0.7	0.6	0.2
d_2	0.8	0.6	0.7	0.3

Non-relevant documents (disliked)

Doc	recommender	system	user	data
d_3	0.2	0.9	0.1	0.8

3) Centroids (average vectors)

	recommender	system	user	data
Relevant centroid C_r	[0.85	0.65	0.65	0.25]
Non-relevant centroid C_{nr}	[0.20	0.90	0.10	0.80]

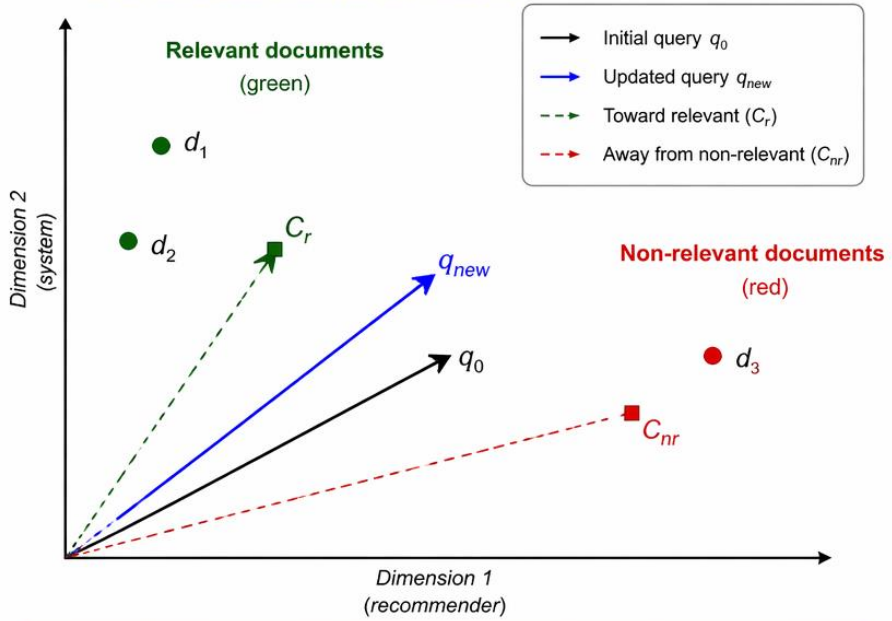
4) Rocchio Update

$$q_{new} = \alpha q_0 + \beta C_r - \gamma C_{nr} \quad (\alpha = 1,\ \beta = 0.75,\ \gamma = 0.25)$$

	recommender	system	user	data
1) q_0	1.00	0.80	0.00	0.00
2) + 0.75 × C_r	+0.64	+0.49	+0.49	+0.19
3) − 0.25 × C_{nnr}	−0.05	−0.23	−0.03	−0.20
q_{new} (updated query)	**1.59**	**1.06**	**0.46**	**0.00***

* negative values are set to 0.

5) Vector Space View (2D for intuition)



6) Interpretation

- "user" weight increased (from 0.00 → 0.46) because it appears in relevant docs.
- "data" weight decreased (from 0.00 → 0.00) because it is emphasized in non-relevant doc.
- Original terms "*recommender*" and "*system*" remain important.

Key Idea

Rocchio moves the query vector toward the centroid of relevant documents and away from the centroid of non-relevant documents.

Result: a better query that focuses on what the user really wants.

Summary and Discussions

- **When to use content-based recommendation?**
  - Ratings of other users are not available
  - Items can be represented by a set of features
  - Users have clear preferences
- **What do we need for generating content-based recommendation?**
  - The content of items
  - The feedback of users (learn their preferences and build the user profile)
- **What are the main tasks of a content-based recommender system:**
  - analyze a set of documents and/or descriptions of items previously rated by the user
  - learn user preferences based on the features of the items rated by the user
  - recommend items that are similar as the user's preferences

Advantages of Content-based Recommendation

	Content-based Recommendation	Collaborative Filtering
User independence	exploits ratings provided by the active user to build his/her own profile	needs ratings from other users to find the nearest neighbors;
Transparency	• *provide explanations clarifying the underlying recommendation algorithm*. E.g., "The action *Movie X has been recommended to you since you liked action movies in the past".* • transparency --> increase **trust**	***Transprency issues:*** only shows the explanation of a recommended item that liked by unknown users/neighbors.
New item	can recommend items that have not yet been rated by any user → resolves the "**first-rater problem**".	the system cannot recommend a new item until it is rated by a number of users.

Disadvantages of Content-based Recommendation

- **Limited content analysis:**
  - They system does not capture sufficiently aspects of the content
  - *E.g.,* A user who likes a *deep, story-driven movie* (e.g., *The Dark Knight*) may be recommended *generic action movies* because the system only captures features such as "genre = action," missing important aspects such as *narrative depth*.
- **Limited diversity**: The system recommends only items similar to items liked by a user before.
- **Over-reliance on user preferences**: The preferences of a user have changed, but the system does not have enough information to recognize it.
- **New-user issues**: The system cannot provide a recommendation for a new user who has not interacted with the system so far.

References

- **[LOP2011]** Lops, P., de Gemmis, M. & Semeraro, G. (2011). *Content-based Recommender Systems: State of the Art and Trends*. In F. Ricci, L. Rokach, B. Shapira & P. B. Kantor (ed.), *Recommender Systems Handbook* (pp. 73-105) . Springer . ISBN: 978-0-387-85819-7.
- **[GLM2015]** De Gemmis M., Lops P., Musto C., Narducci F., Semeraro G. (2015). *Semantics-Aware Content-Based Recommender Systems*. In: Ricci F., Rokach L., Shapira B. (eds) Recommender Systems Handbook. Springer, Boston, MA.
- **[LJM2019]** Lops P., Jannach D., Musto C., Bogers T., and Koolen M. (2019). *Trends in content-based recommendation*. User Modeling and User-Adapted Interaction 29, 239–249.

Exercise 1 – Dice Coefficient

We consider a set of 9 movies, in which each movie is described by a set of categories as shown in the table below.

A user U_a has already watched **M1** and **M4**, and her profile is constructed by combining the categories of these movies.

Which movie is **most relevant** to the user U_a based on the Dice coefficient?

Movie ID	Name	Categories					
		Sci-Fi	Adventure	Drama	Action	Space	Survival
M1	Interstellar	yes	yes			yes	yes
M2	The Notebook			yes			
M3	The Martin	yes	yes			yes	yes
M4	Gravity	yes				yes	
M5	Edge of Tomorrow	yes			yes	yes	
M6	Titanic			yes			yes
M7	Ad Astra	yes	yes				yes
M8	Mad Max: Fury Road		yes		yes		yes
M9	Apollo 13		yes	yes		yes	yes

Exercise 2 – TF-IDF

- Document D contains the following terms with their frequencies:
  - TF(machine, D) = 5
  - TF(learning, D) = 4
  - TF(data, D) = 3
  - TF(algorithm, D) = 2
- Assume document D belongs to a collection of **800 documents**. The document frequencies of the terms are the following:
  - Df(machine) = 100
  - Df(learning) = 400
  - Df(data) = 40
  - Df(algorithm) = 20

Which term has the highest TF-IDF weight in document D?

Exercise 3 – Multivariate Bernoulli Event Model

- We consider a news recommendation system where each article is represented using five binary features: **Politics, Economy, Technology, Health, Sports**
- Each feature takes value:
  - **1** → the topic appears in the article
  - **0** → otherwise
- Articles are labeled as:
  - **1** = Relevant (interesting to the user)
  - **0** = Irrelevant

Article_id	Politics	Economy	Technology	Health	Sports	Class
1	1	1	0	0	0	1
2	0	0	0	1	1	0
3	1	0	1	0	0	1
4	1	1	1	0	0	1
5	0	0	0	1	1	0
6	1	0	1	1	0	?

- The provided table shows five labeled training articles (Article 1–5) and one new article (Article 6) whose class is unknown.
- *Question: Is Article 6 is relevant to the user?*

Rocchio Method

- A user searches for documents using the term space *(AI, data, user, model)*. The initial query focuses on **AI and data**: $q_0 = [1.0, 0.5, 0.0, 0.0]$.
- After reviewing the results, the user marks some documents as relevant and others as non-relevant. The query is then updated using the Rocchio method.
- Relevant document marked by the user: $d_1 = [0.8, 0.6, 0.4, 0.2]$ and $d_2 = [0.7, 0.5, 0.6, 0.3]$
- Non-relevant document marked by the user: $d_3 = [0.2, 0.9, 0.1, 0.7]$
- Parameters: $\alpha = 1.0$, $\beta = 0.75$ and $\gamma = 0.25$

What is the new query?

Thank you for your attention!